

Label Alignment, Fair Domain Adaptation and Edge Deployment for Plant Disease Recognition

Nikita S. Kolesnikov^{1,*} and Alla G. Kravets¹

¹ Department of Computer-Aided Design and Search Engineering, Volgograd State Technical University, 28 Lenin Avenue, Volgograd 400005, Russia; kolesnikov.nikitavlg@gmail.com (N.S.K.); allagkravets@yandex.ru (A.G.K.)

* Correspondence: kolesnikov.nikitavlg@gmail.com; ORCID 0009-0008-5480-6137

Abstract

PlantVillage-trained models exceed 95% accuracy under controlled conditions but transfer poorly to field imagery. Three bottlenecks compound this gap: incompatible label vocabularies, visual domain shift, and the resource limits of edge devices. We study each bottleneck for the PlantVillage-to-PlantDoc transfer. We compare three label-space alignment strategies (fuzzy, Sentence-BERT (SBERT) and hybrid), benchmark five unsupervised domain adaptation (UDA) families under a fair protocol with source-validation checkpoint selection, evaluate the two strongest methods on a second field target (PlantWild v2), and measure five architectures on a Raspberry Pi 4 with a Coral Edge Tensor Processing Unit (TPU). DANN (domain-adversarial training) reaches macro-F1 0.246 ± 0.005 on PlantDoc (+9.2 percentage points (pp) over source-only) and 0.238 ± 0.011 on PlantWild v2; CDAN+E (conditional adversarial alignment) cannot be separated from DANN with three seeds; correlation alignment (CORAL) gains +4.2 pp; minimum class confusion (MCC) collapses. A verified 8-bit integer (INT8) MobileNetV1 runs at 4.6 ms (216 frames per second) at 3.36 MiB on the Edge TPU, and INT8 quantization costs at most 0.3 pp of macro-F1 for the deployable architectures. A supervised reference reaches 0.53 on held-out PlantDoc images, so the unsupervised gap remains large.

Keywords: plant disease recognition; domain shift; unsupervised domain adaptation; label-space alignment; PlantVillage; PlantDoc; edge computing; Raspberry Pi; Edge TPU; INT8 quantization

1. Introduction

Automated plant disease recognition can raise agricultural productivity, particularly in smallholder farming contexts where expert agronomists are unavailable [1,2]. Deep learning models trained on PlantVillage [3] reach near-perfect accuracy under controlled, single-leaf-on-uniform-background conditions [4–6], yet several practical barriers stand between such models and field deployment [7,8].

The first barrier is label-space mismatch. Agricultural image datasets are collected and annotated independently, so their class vocabularies are incompatible: PlantVillage uses Crop__Disease labels, whereas PlantDoc [9] follows different naming conventions. Before any cross-dataset evaluation can occur, a shared label vocabulary has to be constructed.

The second is visual domain shift. Even for classes with matching labels, PlantVillage images are captured under controlled conditions, while field images include complex backgrounds, natural lighting variation and occlusion [7,10]. A ResNet-50 trained on PlantVillage reaches macro-F1 = 0.969 on the PlantVillage test set but only 0.154 on PlantDoc over the same 18 classes, a drop of 0.81 in macro-F1 (Table 1).

The third is the edge device. Field deployment typically requires inference on battery-powered, low-cost hardware. A model of the size of ResNet-50 runs at approximately 5 frames per second on a Raspberry Pi CPU, which is insufficient for real-time monitoring.

We do not propose a new training algorithm. Instead, we systematically study the design choices available to a practitioner who wants to transfer plant disease recognition from PlantVillage to field conditions under a fair, protocol-audited evaluation. Figure 1 gives an overview of the study, and its contributions are the following.

1. A controlled comparison of the fuzzy, SBERT and hybrid label-space alignment strategies, evaluated on the same shared class subset.
2. A five-family UDA benchmark under a strict fair protocol: source-validation checkpoint selection and no target test labels at any stage. We document three defects in our own earlier results (test-set checkpoint selection in the domain-adversarial runs, an in-sample supervised upper bound, and a source-only baseline whose stored predictions did not match the archived checkpoints) and provide corrected numbers.
3. A generalizability evaluation on PlantWild v2 [11] (17 shared classes, 2,336 images of a 115-class, 11,488-image web-scraped dataset).
4. A real-device edge benchmark of latency and throughput for five architectures on a Raspberry Pi 4 with a Coral USB Edge TPU.
5. A conversion-fidelity audit with INT8 accuracy measurement. Hand-written Keras re-implementations used for the first TFLite exports produced degenerate models for three of five architectures; a PyTorch→ONNX→TFLite path reproduces the PyTorch predictions and shows that INT8 post-training quantization costs at most 0.3 pp of macro-F1 for the deployable architectures, including the adapted DANN model. We also document that MCC collapses on small imbalanced targets and that EfficientNet-B3 cannot be delegated to the Edge TPU.

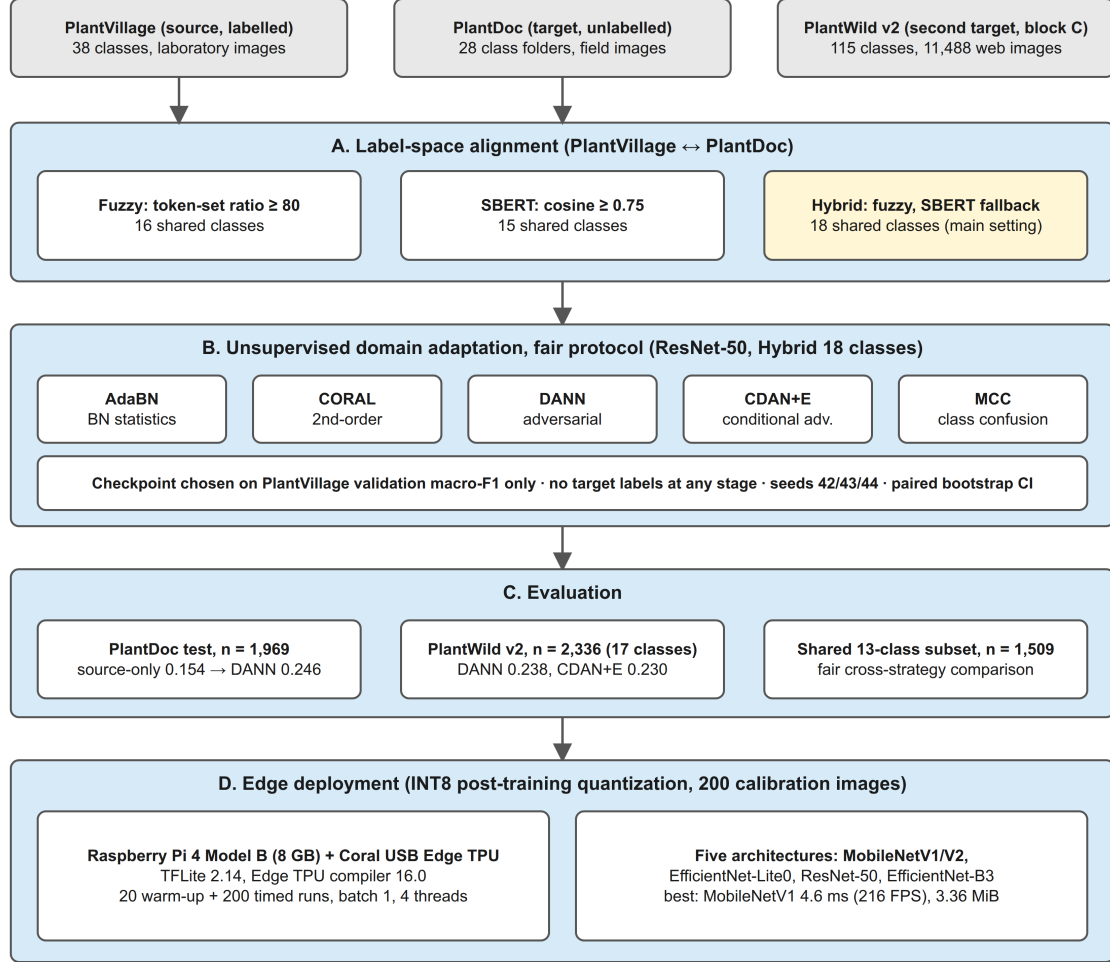


Figure 1: Overview of the study. (A) Three label-space alignment strategies build a shared vocabulary between PlantVillage and PlantDoc. (B) Five unsupervised domain adaptation families are trained under a fair protocol with source-validation checkpoint selection. (C) Evaluation on the PlantDoc test split, on PlantWild v2 as a second field target, and on a shared 13-class subset for a fair cross-strategy comparison. (D) INT8 deployment of five architectures on a Raspberry Pi 4 with a Coral USB Edge TPU.

A companion study by the author [12] benchmarks CORAL, class-weighted CORAL, DANN and CDAN over a larger shared label space with more seeds and characterizes the training collapse of adversarial methods at the standard learning rate. The present paper is complementary: it studies the label-alignment step itself, evaluates AdaBN and MCC in addition, adopts a source-validation model-selection protocol, adds a second field target, and measures deployment cost on edge hardware. Where the two studies overlap (CORAL, DANN, CDAN on PlantDoc), the results are obtained under different protocols; Section 5.4 discusses them jointly.

Section 2 reviews related work. Section 3 describes the datasets, the alignment strategies, the fair evaluation protocol and the edge setup. Section 4 reports the results, Section 5 discusses them and states the limitations, and Section 6 concludes.

2. Related Work

2.1. Plant Disease Recognition under Domain Shift

PlantVillage [3] established the dominant benchmark for automated plant disease recognition, and convolutional networks reach 99% accuracy on its held-out split [4]. Subsequent work documented a severe domain gap when models trained on PlantVillage are evaluated on field imagery: Ferentinos [5] and Too et al. [6] report near-perfect in-domain accuracy, while Barbedo [7] analyses the acquisition

factors (background, lighting, lesion stage) that limit transfer to the field and Toda and Okura [10] visualize which image regions drive the networks’ decisions. PlantDoc [9] provided a challenging field-condition benchmark; the Plant Pathology Challenge [13] and PlantWild [11] added further field-condition data. Surveys of deep learning in agriculture [1,2] identify the laboratory-to-field gap as an open problem. Our best unsupervised result (DANN on PlantDoc: 0.246) is substantially below the held-out supervised reference (macro-F1 0.531, Section 3.4), which reflects the inherent difficulty of the unsupervised setting.

2.2. Unsupervised Domain Adaptation

Classical UDA methods address distribution shift between labeled source and unlabeled target domains [14]. Second-order statistical alignment, Correlation Alignment (CORAL) [15], minimizes the Frobenius distance between covariance matrices; deep adaptation networks match higher-order moments through kernel embeddings [16]. Unconditional adversarial alignment, the Domain-Adversarial Neural Network (DANN) [17], uses a gradient reversal layer; ADDA [18] untangles the two domains with separate encoders. Conditional adversarial alignment, the Conditional Domain Adversarial Network (CDAN) [19], conditions the discriminator on the outer product of features and classifier predictions, optionally weighted by prediction entropy (CDAN+E). Minimum Class Confusion (MCC) [20] minimizes pairwise class confusion on target predictions without a discriminator. Adaptive Batch Normalization (AdaBN) [21] recalibrates running statistics from target images. We evaluate all five families under one fair protocol.

2.3. Agriculture-Specific UDA

Wu et al. [22] propose MSUN (Multi-representation Subdomain Adaptation with Uncertainty Regularization) for PlantVillage→PlantDoc transfer using a ResNet-101 backbone and report 56.06% accuracy on PlantDoc. That figure is not directly comparable with ours: MSUN uses a larger backbone, its own class subset and accuracy rather than macro-F1, and no public code is available. MSUN does not address label-space mismatch or edge deployment. Our study is complementary: it adds label alignment, a five-family controlled comparison under a documented protocol, and edge benchmarking.

Jeon et al. [23] address the lab-to-field gap via background recomposition augmentation followed by unsupervised domain adaptation. Their work targets the same PlantVillage source domain and complementary field targets, providing further evidence that controlled background conditions remain a key source of domain shift. Tunio et al. [24] combine a transformer-fused convolutional backbone with Wasserstein adversarial adaptation on PlantVillage and PlantDoc; Salman et al. [25] apply vision transformers with a mixture of experts to in-the-wild classification; Huan et al. [26] pre-train a unified self-supervised framework on laboratory and field images. These works pursue stronger architectures or pre-training; judging from their published descriptions, none of them reports a model-selection protocol that excludes target test labels, and none reports deployment cost, which is the focus here.

2.4. Cross-Dataset Label-Space Alignment

Label-space mismatch across independently annotated agricultural datasets is rarely addressed. Most UDA works assume a pre-constructed shared taxonomy. Approximate string matching [27] and sentence embeddings [28] are the two natural tools for building one automatically. In the companion study [12] the three resulting label spaces enter only as an experimental factor; here the alignment step itself is the object of study, evaluated on a common class subset under identical downstream conditions.

2.5. Model Compression and Edge Deployment

Post-training 8-bit integer (INT8) quantization [29,30] reduces model size and latency without retraining. The Google Coral Edge Tensor Processing Unit (Edge TPU) accelerates TensorFlow Lite (TFLite)

INT8 models but imposes operator constraints¹; architectures using Swish activations or squeeze-and-excitation blocks require CPU fallback. MobileNetV1/V2 [31,32] and EfficientNet-Lite (the mobile variant of EfficientNet [33], released as a model family without a separate publication) are designed for mobile and embedded inference; ResNet-50 [34] serves as the reference server-class architecture.

3. Materials and Methods

3.1. Datasets

PlantVillage (source). The 38-class controlled-condition leaf disease dataset [3] is used in its segmented form (single leaves on a uniform dark background). After Hybrid alignment it contributes 18 shared classes, 17,195 training images, 3,678 validation images and 3,701 test images. A fixed, class-stratified 10% sample of the training split (1,720 images, shuffled with seed 42, disjoint from validation and test) serves as the calibration set for post-training quantization.

PlantDoc (primary target). This is a field-condition dataset [9]. The original publication describes 27 classes over 13 plant species; the released archive used here contains 28 class folders (the additional folder is `Tomato_two_spotted_spider_mites_leaf`). Hybrid alignment yields 18 shared classes; ten folders receive no mapping and are excluded (Section 3.3). All 1,969 aligned images form the target evaluation set. A class-stratified 80/20 split of this same list (seed 42) provides 1,568 images whose labels are discarded and which serve as the unlabeled adaptation set, and 401 images that are used only as the labeled early-stopping split of the supervised reference (Section 3.4). The adaptation images are therefore a subset of the evaluation images, which is the transductive UDA setting of the standard Office benchmarks [17], whereas the PlantWild v2 evaluation (Section 4.4) is inductive: no PlantWild image is seen before testing.

PlantWild v2 (secondary target). This 115-class web-scraped dataset has 11,488 images [11]. SBERT alignment at threshold 0.65 maps 46 of 115 classes (4,740 images) to the PlantVillage taxonomy; 17 of these classes overlap the 18-class output head of the Hybrid model (`Pepper , _bell___healthy` has no counterpart). Evaluation uses the 2,336 images of those 17 classes.

3.2. Problem Formulation

Let X denote the image space. The source domain $D_S = \{(x_i, y_i)\}$ consists of labeled PlantVillage images with label space Y_S (38 classes); the target domain $D_T = \{x_j\}$ consists of unlabeled target images, of which the adaptation split alone is used during training; the shared label space Y_{ST} , a subset of $Y_S \cap Y_T$, is constructed by label-space alignment. Label-space alignment and feature-level adaptation are two distinct, sequential operations: alignment produces the class vocabulary and adaptation reduces the feature distribution gap. They are not jointly optimized.

3.3. Label-Space Alignment

All class names are normalized to lowercase with underscores replaced by spaces, and three matching strategies are compared. Fuzzy matching uses the token-set ratio [27] with threshold ≥ 80 and yields 16 aligned classes. SBERT matching uses the cosine similarity between `all-MiniLM-L6-v2` sentence embeddings [28] with threshold ≥ 0.75 and yields 15 classes. The Hybrid strategy applies fuzzy matching first and SBERT as a fallback, and yields 18 classes.

The SBERT threshold is a stable operating point rather than a tuned optimum: in the threshold sweep of Table S3, downstream macro-F1 is invariant for thresholds between 0.50 and 0.75 and decreases only at 0.80. For PlantWild v2, whose class names follow the same `{crop} {disease}` convention but include many crops absent from PlantVillage, we report alignment at 0.65, the lower end of the plateau,

¹Model requirements of the Edge TPU compiler: <https://coral.ai/docs/edgetpu/models-intro/#model-requirements> (accessed on 12 September 2026).

to maximize coverage; the downstream evaluation set is defined by the 17 overlapping classes and does not depend on this choice.

For the main UDA benchmark (Tables 2 and 4) we use the Hybrid strategy (18 classes) for a consistent, fixed label space: it is the largest of the three label spaces (a class pair is accepted if either criterion matches it) and was fixed before the UDA runs. On the 13-class subset shared by all strategies its source-only baseline is the lowest of the three (Table 3), so the choice affects the absolute level of the target scores, not the adaptation effects, which are measured within one label space. Fuzzy and SBERT ablations use the same backbone and protocol. Ten PlantDoc folders receive no mapping under any strategy: nine “leaf only” folders without a disease label (Apple_leaf, Blueberry_leaf, Cherry_leaf, Peach_leaf, Raspberry_leaf, Soyabean_leaf, Strawberry_leaf, Tomato_leaf, grape_leaf) and Tomato_two_spotted_spider_mites_leaf, whose PlantVillage counterpart Tomato__Spider_mites Two-spotted_spider_mite is not matched because the normalized names share too few tokens (fuzzy) and fall below the SBERT threshold; the pair is a known false negative of the automatic alignment. The full mappings are provided in Table S6.

3.4. Adaptation Methods and Training Protocol

Models are trained on the labeled source data D_S restricted to the shared classes Y_{ST} . During adaptation the unlabeled target adaptation images (a subset of the evaluation set, Section 3.1) are used for feature alignment only; target labels are never used at any stage of adaptation or checkpoint selection. The checkpoint is selected by maximizing source-domain validation macro-F1 on the 3,678 PlantVillage validation images, so no target information enters model selection. Throughout the paper, differences in macro-F1 are given in percentage points (pp; 1 pp = 0.01 macro-F1).

This protocol corrects an earlier result of ours. A prior DANN run ($F1 = 0.277 \pm 0.009$) used target test labels for per-epoch checkpoint selection, which is a protocol violation. Under the fair protocol DANN reaches $F1 = 0.246 \pm 0.005$ (3 seeds; seed 42: 0.240). All results in this paper use the fair protocol; the legacy result is retained for transparency only.

Five UDA families are compared. AdaBN [21] updates the batch normalization running statistics from target images without any gradient step. CORAL [15] aligns the second-order feature covariance matrices with $\lambda_{CORAL} = 0.1$, which was not tuned on the target. DANN [17] performs unconditional adversarial alignment through gradient reversal. CDAN+E [19] performs conditional adversarial alignment on the outer product of features and predictions with entropy conditioning. MCC [20] minimizes target class confusion with temperature $T = 2.5$.

The source-only models are ResNet-50 [34] networks (ImageNet pretrained) fine-tuned on the aligned PlantVillage training split with AdamW ($lr = 3 \times 10^{-4}$, weight decay 10^{-4}), batch 32, a cosine learning-rate schedule, up to 30 epochs with early stopping (patience 5) on validation macro-F1, input 224×224 , ImageNet normalization and training-time augmentation (horizontal flip, $\pm 15^\circ$ rotation, color jitter 0.2), with seed 42. Table 1 and the source-only predictions used for the bootstrap tests are obtained by re-evaluating the archived checkpoints, the same files that initialized all UDA runs and the edge exports, with the evaluation transform described above; the figures recorded at training time differ slightly and are discussed in Section 5.5.

All five UDA methods start from the same ImageNet-pretrained ResNet-50 and use AdamW ($lr = 3 \times 10^{-4}$, $wd = 10^{-4}$), 20 epochs, batch 32, seeds {42, 43, 44} and identical data pipelines, with source-validation macro-F1 as the model-selection criterion. The loss implementations were unit-tested (20 tests, all passing). For the adversarial methods each source batch of 32 labeled images is paired with a target batch of 32 unlabeled images drawn cyclically from the PlantDoc adaptation split (the target iterator restarts when exhausted); one epoch is one pass over the source loader. The domain discriminator is a three-layer multilayer perceptron (MLP) on the 2,048-dimensional pooled features ($2048 \rightarrow 1024 \rightarrow 512 \rightarrow 2$, ReLU, dropout 0.5). The gradient reversal coefficient follows the standard schedule $\alpha = 2/(1 + \exp(-10p)) - 1$, where $p \in [0, 1]$ is the fraction of training completed; the total loss is the source cross-entropy plus the unweighted domain cross-entropy on source and target batches. CDAN+E feeds

the discriminator the outer product of features and softmax predictions ($2,048 \times 18$), reduced to 2,048 dimensions by a random Gaussian projection matrix that is drawn once under the run seed (42, 43 or 44), stored as a non-trainable buffer and kept fixed, and weights each sample’s domain loss by $\exp(-H(p))$, the entropy-conditioning term. CORAL adds $\lambda = 0.1$ times the squared Frobenius distance between source and target feature covariances to the source cross-entropy; MCC uses temperature 2.5; AdaBN recomputes batch-normalization statistics on the target adaptation images with no gradient step.

The supervised reference is a ResNet-50 initialized from the source-only Hybrid model and fine-tuned on the 1,568 labeled PlantDoc adaptation images (the same images used, without labels, for adaptation), with the backbone frozen for the first 3 epochs, $\text{lr} = 3 \times 10^{-5}$, early stopping on the 401-image split, seed 42, a single run. Because the 1,568 training images belong to the 1,969-image evaluation set, a score on the full evaluation set is an in-sample figure (our earlier reports gave 0.852 on this basis; the archived checkpoint re-evaluates at 0.895 on the full list and at 0.985 on its own training images). We therefore report the held-out estimate on the 401 images the model never trained on: macro-F1 0.531 (accuracy 0.581). These 401 images also served for early stopping, so the estimate is mildly optimistic and, with 7–45 images per class, noisy; a supervised reference on a fully disjoint split requires a re-split and retraining and is left for future work.

The edge models (MobileNetV1, MobileNetV2, EfficientNet-Lite0, EfficientNet-B3 and ResNet-50) are each fine-tuned on the same aligned PlantVillage split with the source-only recipe above (18-class head, no domain adaptation) and then quantized (Section 3.5). Their purpose is to measure deployment cost per architecture; their target-domain accuracy is not part of the benchmark.

Training ran on a single workstation GPU (NVIDIA GeForce RTX 5070 Ti, 16 GB) under Linux with Python 3.12 and PyTorch 2.x; one UDA run (20 epochs) takes about 20–25 min. Per-run configuration, git revision and seeds are stored alongside every result file.

3.5. Evaluation Metrics and Edge Measurement Setup

The primary metric is macro-F1; accuracy and balanced accuracy are secondary. Statistical uncertainty is reported as the paired bootstrap 95% confidence interval (CI) [35] of the difference in macro-F1 against source-only on the 1,969 PlantDoc test predictions of seed 42 (2,000 resamples). For the direct DANN versus CDAN+E comparison we use 10,000 resamples per seed to resolve the smaller differences. Multiple testing is not Bonferroni-corrected; five comparisons against source-only and three seed-wise comparisons are reported in full.

Edge measurements use a Raspberry Pi 4 Model B (8 GB RAM) with a Coral USB Edge TPU on USB 3.0, the Edge TPU compiler v16.0.384591198, TFLite runtime v2.14.0 on Debian 12 (Bookworm), Python 3.11.2 and libedgetpu1-std 16.0. Each model is measured with 20 warm-up runs and 200 measured runs at batch 1 with 4 CPU threads; each timed run covers one interpreter invocation on a fixed synthetic input tensor (seed 42), so image decoding and preprocessing are excluded. We report mean latency, frames per second and model size (Table 5); latency standard deviation and p95 are given in Table S4. Model sizes are file sizes in MiB (2^{20} bytes).

All architectures use post-training INT8 quantization (PTQ) with 200 representative calibration images from PlantVillage (the first 200 rows of the shuffled, class-stratified calibration set defined in Section 3.1; all 18 classes are present, with 2–46 images per class in the proportions of the training split rather than balanced).

Two conversion paths were used. The first TFLite exports were produced by hand-written Keras re-implementations of each architecture into which the PyTorch weights were copied layer by layer (the “Keras-rebuild” path). An audit of these files against the PyTorch checkpoints (Section 4.6) showed that three of the five re-implementations were defective, so a second, verifiable path was added: PyTorch $\rightarrow$ ONNX (opset 17; for ResNet-50 the max-pool padding is expressed as an explicit zero pad, which is equivalent after ReLU and avoids an operator the Edge TPU compiler rejects) $\rightarrow$ TensorFlow Saved-Model via onnx2tf 1.20 $\rightarrow$ full-integer INT8 TFLite (int8 input and output, per-channel weights) with

the same 200 calibration images. Table 5 reports the latency of the verified files: the ONNX-path INT8 models for MobileNetV1, MobileNetV2, ResNet-50 and the adapted DANN model, the verified Keras-path model for EfficientNet-Lite0, and the ONNX-path INT8 EfficientNet-B3 (CPU only). Accuracy of both paths is evaluated on the host with the TFLite interpreter on the full PlantVillage and PlantDoc test splits, using one fixed preprocessing (Python Imaging Library (PIL) resize to 224×224 , scaling to $[0, 1]$, ImageNet normalization) for the TFLite models and for the PyTorch reference of the same checkpoint.

4. Results

4.1. Source-Only Baseline

Table 1 reports the zero-shot performance of the ResNet-50 trained on PlantVillage (PV) and evaluated on PlantDoc without adaptation, for each alignment strategy. The values come from the archived checkpoints re-evaluated with the canonical transform (Section 3.4). The strategies differ in class-set size, so the rows are not directly comparable; Table 3 provides the comparison on a common subset.

Table 1: Source-only macro-F1 per alignment strategy.

Strategy	Classes	PV Macro-F1	PlantDoc Macro-F1
Fuzzy	16	0.972	0.187
SBERT	15	0.986	0.171
Hybrid	18	0.969	0.154

4.2. UDA Benchmark on PlantDoc

Table 2 summarizes the UDA benchmark on PlantDoc in the Hybrid label space (18 classes) under the fair protocol. Macro-F1 is the mean $\pm$ sample standard deviation over seeds 42, 43 and 44. The column Δ (3 seeds) is the mean difference to source-only in pp computed from unrounded values (source-only = 0.1539); Δ (seed 42) and its CI are the point estimate and the paired bootstrap 95% CI of the difference on the seed-42 predictions ($n = 1,969$, 2,000 resamples). The two Δ columns have different bases and are not expected to coincide. All methods use a ResNet-50 backbone.

Table 2: UDA benchmark on PlantDoc (Hybrid, 18 classes).

Method	Family	Macro-F1	Δ (3 seeds), pp	Δ (seed 42), pp	95% CI of Δ (seed 42), pp	Outcome
Source-only	—	0.154	—	—	—	—
AdaBN	BN statistics	0.155 ± 0.000	+0.1	+0.1	$[-0.0, +0.4]$	no effect
CORAL	2nd-order	0.196 ± 0.012	+4.2	+4.3	$[+2.5, +6.0]$	better
DANN	Uncond. adv.	0.246 ± 0.005	+9.2	+8.6	$[+6.7, +10.5]$	better
CDAN+E	Cond. adv.	0.242 ± 0.026	+8.8	+11.3	$[+9.4, +13.1]$	better
MCC	Classifier	0.077 ± 0.021	-7.7	-7.7	$[-9.6, -5.7]$	worse (collapse)
Supervised reference	<i>uses PlantDoc labels</i>	<i>0.531 (held-out n = 401, 1 run)</i>	—	—	—	—

Figure 2 visualizes Table 2: panel (a) shows the per-seed spread, panel (b) the bootstrap intervals. For CDAN+E seed 42 is the best of its three seeds (0.267), which is why its seed-42 difference (+11.3 pp) and interval lie above its three-seed mean difference (+8.8 pp).

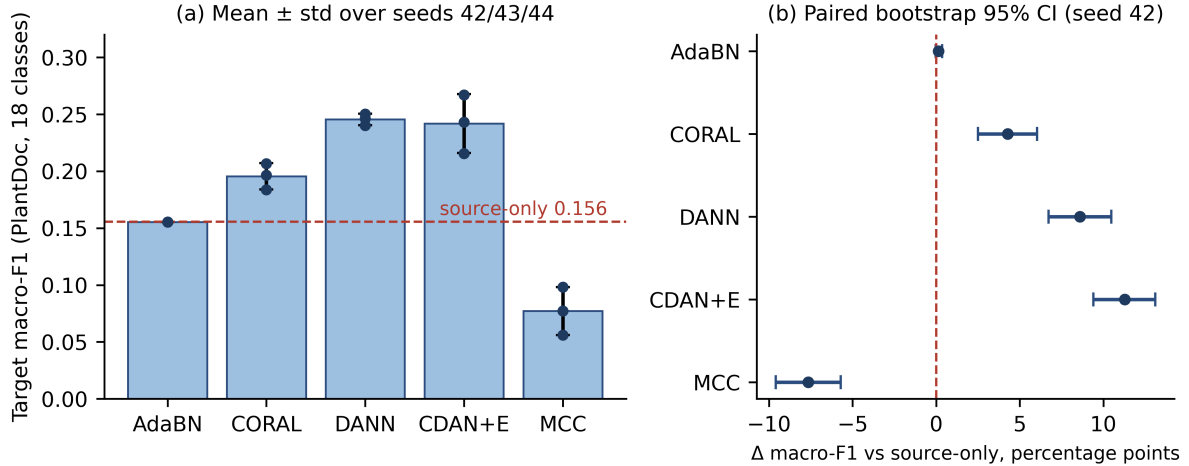


Figure 2: UDA benchmark on PlantDoc (Hybrid 18 classes). (a) Target macro-F1, mean $\pm$ std over seeds 42/43/44 with per-seed points; dashed line: source-only baseline (0.154). (b) Paired bootstrap 95% confidence intervals of the difference to source-only on seed-42 predictions ($n = 1,969$, 2,000 resamples), in percentage points.

DANN achieves the highest mean macro-F1 on PlantDoc (0.246 ± 0.005 , +9.2 pp, bootstrap CI excludes zero), and CDAN+E is close (0.242 ± 0.026 , +8.8 pp). The mean difference between DANN and CDAN+E is 0.004 macro-F1, which is small relative to the across-seed variability of CDAN+E (std = 0.026). DANN also outperforms CORAL (0.246 vs. 0.196); the earlier DANN macro-F1 of 0.277 was inflated by test-set checkpoint selection. MCC collapsed on all three seeds: the class-confusion objective caused source forgetting from the first epoch, source-validation macro-F1 never exceeded 0.28 in any run, and the selected checkpoints score 0.056–0.098 on the target, which confirms its incompatibility with small, imbalanced target datasets at default hyperparameters. AdaBN provides no benefit, so recalibrating batch normalization statistics alone is insufficient for this domain gap.

A per-seed paired bootstrap of DANN against CDAN+E (same $n = 1,969$ test samples, 10,000 resamples) gives inconsistent directionality: on seed 42 CDAN+E is significantly better ($\Delta = -0.027$, CI $[-0.046, -0.006]$); on seed 43 DANN is significantly better ($\Delta = +0.035$, CI $[+0.017, +0.052]$); seed 44 is inconclusive ($\Delta = +0.003$, CI $[-0.016, +0.022]$). The three-seed mean difference (DANN – CDAN+E = +0.004) is small relative to CDAN+E’s across-seed variability. Three seeds therefore do not support a population-level ranking of DANN and CDAN+E on this benchmark; the evidence is inconclusive rather than a demonstrated equivalence. DANN is, however, markedly more stable across seeds (std 0.005 vs. 0.026).

4.3. Fair Cross-Strategy Comparison

To compare the alignment strategies on identical test samples we restrict the evaluation to the 13 classes that all three class mappings share ($n = 1,509$ PlantDoc images). Table 3 re-scores the re-evaluated predictions of the seed-42 source-only models (Section 3.4) and the stored predictions of the earlier single-seed CORAL runs on these classes only; no model was retrained. The CORAL runs use the per-strategy λ from the sweep in Table S1, which was evaluated on the target split under the earlier protocol. With a single seed per cell no standard deviation or confidence interval is available.

Table 3: Baseline and CORAL macro-F1 on the shared 13-class subset.

Strategy	Baseline F1	CORAL F1	Gain
Fuzzy	0.140	0.178	+0.038
SBERT	0.147	0.174	+0.026
Hybrid	0.118	0.142	+0.023

CORAL improves all three strategies on the shared subset by +2.3 to +3.8 pp. The spread between

strategies (1.5 pp) is of the order of CORAL’s seed-to-seed standard deviation in Table 2 (1.2 pp), so with one seed per cell the gains are consistent with being equal, and the control shows that the CORAL gain is not an artifact of class-set size differences. The control was run for CORAL only; the adversarial methods were trained in the Hybrid space alone, so their gains in Table 2 have no cross-strategy counterpart. Figure 3 shows two-dimensional t-distributed stochastic neighbor embedding (t-SNE) projections (perplexity 30, PCA initialization, seed 42) of the ResNet-50 features of the seed-42 models before and after CORAL adaptation in the Hybrid label space.

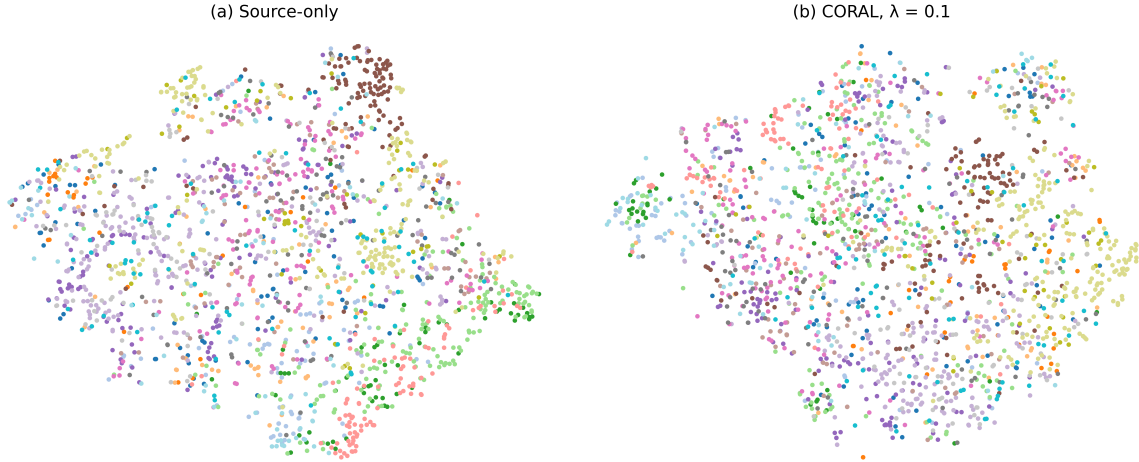


Figure 3: Two-dimensional t-SNE projection (scikit-learn, perplexity 30, PCA initialization, learning rate “auto”, random state 42) of ResNet-50 penultimate-layer features on the PlantDoc evaluation set (Hybrid label space, one color per class): (a) source-only model (seed 42); (b) after CORAL adaptation ($\lambda = 0.1$, seed 42). CORAL tightens several class clusters but substantial overlap remains, consistent with the modest macro-F1 gain in Table 2.

4.4. Generalizability: PlantWild v2

We evaluate the Hybrid model on PlantWild v2 (115 classes, 11,488 web-scraped images; 46 of 115 classes align with PlantVillage at threshold 0.65). Table 4 reports the fair-protocol results with source-validation checkpoint selection and 3 seeds per method; the source-only row is the single seed-42 model. All methods are the 18-class Hybrid models adapted on PlantDoc unlabeled data; macro-F1 is computed over the 17 classes shared with PlantWild v2 ($n = 2,336$), and no adaptation on PlantWild images was performed. Δ src is computed from unrounded values (source-only = 0.1722).

Table 4: PlantWild v2 macro-F1 of the PlantDoc-adapted models.

Method	Macro-F1	Δ src, pp	Seed std
Source-only	0.172	—	—
DANN	0.238 $\pm$ 0.011	+6.6	0.011
CDAN+E	0.230 $\pm$ 0.038	+5.7	0.038

Per seed, DANN scores 0.242, 0.226 and 0.247 (seeds 42, 43, 44) and CDAN+E 0.270, 0.195 and 0.224. Both methods improve over the source-only baseline on PlantWild v2 even though neither model has seen a PlantWild image. DANN achieves a mean macro-F1 of 0.238 ± 0.011 , compared with 0.230 ± 0.038 for CDAN+E. The mean difference (+0.85 pp for DANN, from unrounded means) is small relative to CDAN+E’s seed variance; the ordering $\text{DANN} \geq \text{CDAN+E}$ and the stability advantage of DANN are the same as on PlantDoc.

Label alignment on BeanDiseases (3 disease-only labels, no crop prefix) yielded no matches at threshold 0.65; the nearest SBERT match was `bean_rust` $\rightarrow$ `Apple Cedar apple rust` at cosine = 0.63. Datasets sharing PlantVillage’s {crop} {disease} naming convention (PlantDoc, PlantWild v2) align successfully, while disease-only label schemes require format adaptation.

4.5. Edge Deployment: Latency and Throughput

Table 5 reports the latency of the INT8 PTQ models with Hybrid 18-class heads on the Raspberry Pi 4 CPU (TFLite, 4 threads) and on the Coral USB Edge TPU, from 200 timed runs after 20 warm-up runs. Size is the INT8 TFLite file size in MiB (CPU file; the Edge TPU-compiled file differs by less than 0.5 MiB), and FPS is computed from the unrounded mean latency (e.g. 4.62 ms $\rightarrow$ 216 FPS). The measured files are the verified INT8 models of Section 3.5 (ONNX path; EfficientNet-Lite0 via the verified Keras path); each cell is one 200-run session, and standard deviations and p95 are in Table S4. An earlier measurement of the first-generation files, three of which later proved to carry defective weights, gave 4.6 / 5.9 / 6.3 / 56.0 ms on the Edge TPU for the first four rows; the agreement confirms that latency is a property of the operator graph, not of the weight values.

Table 5: Edge latency, throughput and model size on Raspberry Pi 4 with Coral Edge TPU.

Architecture	Params	CPU (ms)	Edge TPU (ms)	FPS (Edge TPU)	Size (MiB)
MobileNetV1	3.2M	32.9	4.6	216	3.36
MobileNetV2	2.2M	30.4	5.1	197	2.60
EfficientNet-Lite0	3.5M	33.0	6.3	159	3.79
ResNet-50	23.5M	190.3	55.2	18	23.14
ResNet-50 DANN (seed 42)	23.5M	190.1	54.9	18	23.14
EfficientNet-B3	10.8M	137.7	—	—	11.93

MobileNetV1 achieves 4.6 ms (216 FPS) on the Edge TPU at 3.36 MiB, well within a real-time budget, and MobileNetV2 and EfficientNet-Lite0 are within 2 ms of it. The adapted DANN model has the same operator graph as the source-only ResNet-50 and the same latency (54.9 versus 55.2 ms), so domain adaptation costs nothing at inference time. On the CPU the three small networks are within 3 ms of each other (30–33 ms), and ResNet-50 is about six times slower. EfficientNet-B3 cannot be delegated to the Edge TPU because Swish activations and squeeze-and-excitation blocks are not supported by the compiler.

4.6. Conversion Fidelity and INT8 Accuracy

INT8 PTQ reduces the ResNet-50 TFLite model from 89.7 MiB (FP32) to 23.1 MiB (3.9 $\times$). Whether the quantized models still recognize diseases is a separate question, which we answer by evaluating every converted model on the full test splits and comparing it with the PyTorch checkpoint it was derived from. Table 6 gives the macro-F1 of the converted models on the PlantVillage (PV, $n = 3,701$) and PlantDoc (PD, $n = 1,969$) test splits in the Hybrid label space. The reference is the PyTorch FP32 model of the same checkpoint evaluated with the same PIL preprocessing as the TFLite models (PIL bicubic resize). Elsewhere in the paper models are evaluated with the torchvision transform (bilinear resize with antialiasing), which differs by 1–2 pp: under that transform the source-only checkpoint gives 0.969 / 0.154 (Table 1) and DANN seed 42 gives 0.947 / 0.240, versus 0.974 / 0.148 and 0.961 / 0.249 here. The difference is the resize implementation, not the conversion. “Keras-rebuild” denotes the hand-written re-implementations of the first exports and “ONNX $\rightarrow$ TFLite” the verifiable path of Section 3.5; Δ is given in pp relative to the reference, and the last column lists the operations mapped by Edge TPU compiler 16.0.

Table 6: Macro-F1 of the converted models against their PyTorch checkpoints.

Model	PyTorch FP32 PV / PD	Keras-rebuild INT8 PV / PD	ONNX $\rightarrow$ TFLite INT8 PV / PD	Δ INT8 (ONNX path) PV / PD	Edge TPU
MobileNetV1	0.989 / 0.124	0.102 / 0.029	0.989 / 0.122	−0.02 / −0.26	34 / 34 ops
MobileNetV2	0.976 / 0.134	0.004 / 0.009	0.976 / 0.142	+0.02 / +0.79	69 / 69 ops

Model	PyTorch FP32 PV / PD	Keras-rebuild INT8 PV / PD	ONNX→TFLite INT8 PV / PD	Δ INT8 (ONNX path) PV / PD	Edge TPU
EfficientNet-Lite0	0.982 / 0.212	0.982 / 0.205	—	−0.09 / −0.69 (Keras path)	60 / 60 ops (Keras path)
ResNet-50 (source-only)	0.974 / 0.148	0.946 / 0.128	0.971 / 0.149	−0.22 / +0.11	77 / 77 ops
ResNet-50 DANN (seed 42)	0.961 / 0.249	0.786 / 0.184	0.960 / 0.247	−0.11 / −0.19	77 / 77 ops
EfficientNet-B3	0.984 / 0.255	0.008 / 0.005	0.958 / 0.146	−2.55 / −10.95	not compilable

Four findings follow from Table 6.

1. The first-generation exports were unusable for MobileNetV1, MobileNetV2 and EfficientNet-B3: their INT8 files score at chance level on PlantVillage, and the defect reproduces when the current checkpoints are pushed through the same scripts, so it lies in the re-implementations, not in the weights. The Keras-rebuild ResNet-50 loses 2–3 pp because the Keras reference architecture places the stride-2 convolution in a different position than torchvision; EfficientNet-Lite0 was the only correct re-implementation.
2. With a faithful conversion, INT8 quantization is nearly free for the deployable architectures. The float32 TFLite models reproduce the PyTorch predictions (top-1 agreement 0.99–1.00 on PlantVillage), and full-integer quantization lowers macro-F1 by at most 0.3 pp for MobileNetV1, MobileNetV2 and ResNet-50 on both test splits (full audit in Table S5); the one larger change, +0.8 pp for MobileNetV2 on PlantDoc, is a gain within the noise of a 1,969-image test set. The adapted DANN model survives quantization equally well (PlantDoc 0.247 versus 0.249), so the domain-adaptation gain is preserved on the integer model that would actually be deployed.
3. EfficientNet-B3 does not quantize well under post-training INT8 (−2.6 pp on PlantVillage, −11 pp on PlantDoc) and, being non-delegable, offers no advantage on this hardware.
4. The backbone may matter as much as the adaptation method. The PyTorch column of Table 6 doubles as a source-only comparison of architectures on PlantDoc: EfficientNet-B3 reaches macro-F1 0.255 and EfficientNet-Lite0 0.212 without any adaptation, whereas the adapted ResNet-50 (DANN) reaches 0.249 under the same preprocessing (0.246 canonical). This is a single-seed, single-run observation without a confidence interval, whereas the DANN figure is a three-seed mean, so it is indicative rather than conclusive. The UDA benchmark of Section 4.2 is confined to ResNet-50, so this does not contradict the adaptation gains, which are measured within one backbone, but it suggests that a better-pretrained, more modern backbone can recover as much of the field gap as adversarial adaptation does on ResNet-50. Combining the two was outside the scope of this study and is the most promising follow-up.

The four ONNX-path INT8 models compile with every operation mapped to the Edge TPU, and Table 5 reports their measured latency. Relative to the first-generation graphs, the verified MobileNetV1 and MobileNetV2 graphs differ by explicit padding operators (34 versus 29 and 69 versus 70 operations) and the verified ResNet-50 is the torchvision v1.5 variant rather than the Keras v1 graph; on the Edge TPU the differences are 0.0, −0.8 and −0.8 ms respectively, so the verified files are as fast as or slightly faster than the defective ones, and the ranking is unchanged.

5. Discussion

5.1. What Helps

Adversarial alignment gives the largest gains on both field targets. DANN reaches 0.246 ± 0.005 (+9.2 pp) on PlantDoc and 0.238 ± 0.011 (+6.6 pp) on PlantWild v2; CDAN+E is close on both (0.242 ± 0.026 , +8.8 pp; 0.230 ± 0.038 , +5.7 pp). Both outperform CORAL (0.196 ± 0.012 on PlantDoc). DANN is ahead of CORAL under the fair protocol on PlantDoc (0.246 vs. 0.196, both bootstrap-significant against source-only). DANN and CDAN+E cannot be separated with three seeds, and DANN is the more stable of the

two (std 0.005 vs. 0.026 on PlantDoc, 0.011 vs. 0.038 on PlantWild v2).

CORAL is the safest low-cost option. It has no discriminator, no adversarial schedule, one hyperparameter that, in the fair benchmark, was not tuned on the target ($\lambda = 0.1$), and a modest but consistent gain (Table 3: +2.3 to +3.8 pp across all three label spaces on the shared subset, earlier protocol, single seed). Its seed spread (0.012) is larger than DANN’s, but its training never failed in any configuration here or in the companion study [12].

5.2. What Does Not Help

AdaBN provides no benefit (F1 = 0.155 vs. 0.154 source-only): recalibrating batch normalization statistics is insufficient for this domain gap. MCC failed on all three seeds, because the class-confusion objective caused source forgetting from the first training epoch on our small, imbalanced target dataset (1,568 images, 18 unbalanced classes); MCC is not suitable for this deployment scenario without significant hyperparameter adaptation. The label-alignment strategy does not strongly predict downstream UDA performance on the shared 13-class subset: CORAL improves all three strategies by +2.3 to +3.8 pp (single seed per cell, earlier protocol), which rules out class-set-size artifacts.

5.3. Gap Analysis

The held-out supervised reference (macro-F1 0.531 on 401 images, Section 3.4) quantifies the gap to the best unsupervised result (DANN on PlantDoc: 0.246): about 0.29 in macro-F1, so DANN recovers about a quarter of the distance from source-only (0.154) to the supervised reference. Among the evaluated methods under the present protocol, target macro-F1 remains below 0.25. The supervised reference uses PlantDoc labels and is not directly comparable to the UDA results; it is a single run on a small held-out split and should be read as an order of magnitude, not a precise ceiling. The in-sample figure of 0.852 reported in our earlier versions was measured on images the model had been trained on and is withdrawn.

For an agronomic reading, the per-class scores in Table S2 and the confusion matrices in Figure S1 matter more than the macro average: on PlantDoc the source-only model recognizes squash powdery mildew, grape black rot, potato early and late blight and corn common rust best, and fails completely on tomato bacterial spot and tomato mosaic virus; after CORAL, tomato yellow leaf curl virus becomes the best-recognized class, while several classes lose recall. Per-crop deployment decisions should therefore be taken class by class, not from the macro-F1 alone.

For the practitioner the consequence is clear: at macro-F1 ≈ 0.25 over 18 classes, none of the unsupervised models is fit to act as a stand-alone field diagnostic. What the adapted models can support today is triage, that is, flagging leaves for human or laboratory confirmation and prioritizing field images for annotation, and the latency results of Section 4.5 establish only that on-device compute is not the limiting factor for such a workflow. Closing the accuracy gap, not the latency budget, is the bottleneck for deployment.

5.4. Relation to the Companion Study

The companion study [12] and the present paper share the source dataset, the backbone and three of the adaptation methods, yet they reach different conclusions about adversarial adaptation: there, DANN and CDAN collapse to near-random performance in a majority of runs at the standard learning rate and CORAL is recommended as the default; here, DANN is the best-performing method with no collapsed run and CDAN+E is competitive. Three protocol differences separate the two experiments, and we state them rather than adjudicate between them. The companion study is under review at the time of writing, so the comparison is made at the level of its stated conclusions and protocol, and no figure from it is reproduced here.

The first difference is model selection. The companion study evaluates a fixed training endpoint, whereas here the checkpoint is chosen on PlantVillage validation macro-F1. The per-epoch validation

history of the adversarial runs is not monotone: for DANN seed 42 the source-validation macro-F1 drops to 0.31–0.44 at epochs 4–6 and again at epochs 19–20, and the selected checkpoint is epoch 7. A source-validation criterion therefore skips exactly the collapsed epochs that a fixed-endpoint protocol would report. This selection uses no target information and is legitimate under the fair protocol, but it explains why the same algorithm can appear unstable in one protocol and stable in another. The second difference is the label space: the companion study uses a larger shared label space than the 18 Hybrid classes used here, and macro-F1 over a larger class set with rarer classes is both harder and noisier. The third is the number of seeds: the companion study uses more seeds than the three used here, and three seeds cannot rule out an occasional collapse that a larger number would reveal.

The two studies are therefore consistent on the points where they overlap: CORAL is a robust default that never fails, adversarial methods can achieve higher macro-F1 but their outcome depends on the training-time selection criterion, and neither closes more than about a quarter of the gap to the supervised reference. The practical recommendation that follows is protocol-specific: with a validation-based checkpoint selector in place, DANN is the better choice; without one, CORAL is.

5.5. Scope and Limitations

All experiments use PlantVillage as the single source domain, and the alignment pipeline depends on {crop} {disease} naming: BeanDiseases, with disease-only labels, yields 0 aligned classes. The 18 class correspondences were constructed automatically and inspected by the author; they were not validated by an independent plant pathologist. Means and standard deviations rest on three seeds per method (the companion study uses five), and the bootstrap intervals in Table 2 describe sampling uncertainty over test images for one seed, not seed-to-seed variability. MCC was run with default hyperparameters, which are unsuitable for small, imbalanced targets; a tuned MCC might behave differently. All five adaptation methods were run on ResNet-50 only; Table 6 shows that a source-only EfficientNet-B3 already matches the adapted ResNet-50 on PlantDoc, so the ranking of methods may change with a stronger backbone.

The evaluation protocol on PlantDoc is transductive: the unlabeled adaptation images are a subset of the evaluation set (Section 3.1). This is the usual UDA setting and no target label is ever used, but the PlantDoc numbers are not inductive estimates, whereas the PlantWild v2 results, obtained without seeing any PlantWild image, are. The supervised reference of 0.531 is one fine-tuning run (seed 42) evaluated on 401 held-out images that also served for early stopping; the 0.852 of our earlier versions was an in-sample score on the model’s own training images and is withdrawn, and a clean supervised reference needs a re-split of PlantDoc with a disjoint test set. The legacy DANN result ($F1 = 0.277 \pm 0.009$) used target-test checkpoint selection; the fair-protocol results ($F1 = 0.246 \pm 0.005$) are the canonical numbers, and all conclusions use the fair protocol.

Three provenance issues concern the measurements themselves. First, the source-only figures recorded when the models were trained (PlantVillage / PlantDoc macro-F1 0.977 / 0.187 for Fuzzy, 0.975 / 0.152 for SBERT, 0.956 / 0.156 for Hybrid) do not reproduce from the archived checkpoints, and the stored Hybrid predictions agree with the re-evaluation on only 40% of PlantDoc images, whereas every fair-protocol UDA checkpoint reproduces its recorded score exactly; the source-only checkpoint files were evidently re-saved after those predictions were recorded. Because the archived checkpoints are the ones that initialized all UDA runs and the edge exports, Table 1, the shared-subset control (Table 3) and the bootstrap baseline of Table 2 use their re-evaluation (Hybrid 0.969 / 0.154); relative to the recorded figures this moves the PlantDoc baseline by -0.16 pp for Hybrid, -0.01 pp for Fuzzy and $+1.85$ pp for SBERT, and no conclusion changes. Both sets of predictions are included in the data deposit. Second, Table 5 is one 200-run session per file on one device; run-to-run variation on the CPU was up to 9 ms for ResNet-50 (standard deviations in Table S4), and a repeated CPU session for the same ResNet-50 file differed by tens of milliseconds when the board had just finished another benchmark, so CPU figures should be read to about $\pm 10\%$, while Edge TPU figures were stable to within 0.6 ms. Third, INT8 accuracy was measured on the host: the TFLite interpreter on x86 executes the same integer kernels,

but Edge TPU execution of the four compiled models was not verified end to end.

6. Conclusions

We presented a systematic empirical study of three practical bottlenecks in cross-dataset plant disease recognition, namely label-space mismatch, visual domain shift and edge-device constraints, using PlantVillage as source and PlantDoc and PlantWild v2 as field targets.

Label alignment mainly determines class coverage, not adaptation quality. Fuzzy, SBERT and hybrid matching yield 16, 15 and 18 shared classes; on a common 13-class subset CORAL improves all three by +2.3 to +3.8 pp (single seed per cell, earlier protocol), so the choice of strategy should be driven by coverage and naming conventions of the target dataset.

Under a fair, source-validation protocol, adversarial adaptation gives the largest gains. DANN reaches macro-F1 0.246 ± 0.005 on PlantDoc (+9.2 pp over source-only) and 0.238 ± 0.011 on PlantWild v2 (+6.6 pp) without seeing a PlantWild image; CDAN+E cannot be separated from DANN with three seeds but is 3.5–5 times more variable across them (std ratio 5.2 on PlantDoc, 3.5 on PlantWild v2); CORAL gives a smaller, reliable +4.2 pp; AdaBN gives nothing and MCC collapses.

The protocol matters as much as the method. Our own earlier DANN result (+12.4 pp against the re-evaluated baseline) was inflated by test-set checkpoint selection, and our earlier supervised upper bound (0.852) was an in-sample score; the corrected figures are +9.2 pp and 0.531. The disagreement with the companion study about adversarial stability is traceable to the checkpoint-selection criterion, the class-space size and the number of seeds (Section 5.4).

The unsupervised gap remains large, and the backbone is part of the answer. The best UDA result recovers about a quarter of the distance from source-only (0.154) to the held-out supervised reference (0.531); in a single-seed comparison a source-only EfficientNet-B3 reaches the same level (0.255) as the adapted ResNet-50 without any target data, so backbone choice and domain adaptation should be studied jointly rather than in isolation.

On-device compute is not the bottleneck; accuracy is. The latency budget for real-time inference is met on commodity edge hardware: a verified INT8 MobileNetV1 runs at 4.6 ms per image (216 FPS) at 3.36 MiB on a Coral Edge TPU attached to a Raspberry Pi 4; MobileNetV2 and EfficientNet-Lite0 are within 2 ms of it, ResNet-50 is 12× slower, the adapted DANN model is exactly as fast as its source-only counterpart, and EfficientNet-B3 cannot be delegated to the accelerator. With a verifiable PyTorch→ONNX→TFLite conversion, full-integer INT8 quantization costs at most 0.3 pp of macro-F1 for MobileNetV1, MobileNetV2 and ResNet-50, and the adapted DANN model keeps its PlantDoc macro-F1 after quantization (0.247 versus 0.249). The first-generation hand-written Keras exports, by contrast, were unusable for three of the five architectures, which argues for verified conversion tooling in any deployment pipeline; re-timing the verified files reproduced the first-generation latencies to within 1 ms, confirming that latency is a property of the graph.

All code, configurations, per-seed predictions and result files are released (Data Availability Statement).

Declarations

Supplementary Materials: The following supporting information is provided as a supplementary file with the submission and in the repository listed in the Data Availability Statement: Table S1, CORAL λ sweep for all three label spaces; Table S2, per-class F1 for the 18 Hybrid classes (source-only and CORAL); Table S3, SBERT threshold sweep (E1); Table S4, full edge benchmark metrics (latency standard deviation and p95, CPU load, memory, temperature) for all architecture × strategy × backend combinations; Table S5, complete conversion-fidelity audit (all variants, both splits, top-1 agreement); Table S6, the three class mappings; Figure S1, confusion matrices on PlantDoc.

Author Contributions: Conceptualization, N.S.K. and A.G.K.; methodology, software, validation, formal analysis, investigation, data curation, writing—original draft preparation and visualization,

N.S.K.; writing—review and editing, N.S.K. and A.G.K.; supervision, A.G.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: PlantVillage, PlantDoc, PlantWild v2 and BeanDiseases are publicly available third-party datasets. The code (training, adaptation, quantization and benchmarking scripts), configuration files, class mappings, data splits, per-run result files with git revisions, per-seed test predictions, bootstrap outputs, the conversion-fidelity and supervised-reference audits and the edge benchmark logs supporting the reported results are archived at Zenodo (<https://doi.org/10.5281/zenodo.22734464>, version 1.0.0) and maintained at <https://github.com/Fgeeha/Empirical-Study-of-Label-Space> (directories results/uda_benchmark/, results/final/, results/int8_accuracy/, results/ub_audit/, results/edgebench/, configs/, splits/, experiments/). PlantWild is used in its v2 release.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Kamilaris, A.; Prenafeta-Boldú, F.X. Deep learning in agriculture: A survey. *Comput. Electron. Agric.* **2018**, *147*, 70–90. <https://doi.org/10.1016/j.compag.2018.02.016>
2. Liu, J.; Wang, X. Plant diseases and pests detection based on deep learning: A review. *Plant Methods* **2021**, *17*, 22. <https://doi.org/10.1186/s13007-021-00722-9>
3. Hughes, D.P.; Salathé, M. An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv* **2015**, arXiv:1511.08060. <https://doi.org/10.48550/arXiv.1511.08060>
4. Mohanty, S.P.; Hughes, D.P.; Salathé, M. Using deep learning for image-based plant disease detection. *Front. Plant Sci.* **2016**, *7*, 1419. <https://doi.org/10.3389/fpls.2016.01419>
5. Ferentinos, K.P. Deep learning models for plant disease detection and diagnosis. *Comput. Electron. Agric.* **2018**, *145*, 311–318. <https://doi.org/10.1016/j.compag.2018.01.009>
6. Too, E.C.; Yujian, L.; Njuki, S.; Yingchun, L. A comparative study of fine-tuning deep learning models for plant disease identification. *Comput. Electron. Agric.* **2019**, *161*, 272–279. <https://doi.org/10.1016/j.compag.2018.03.032>
7. Barbedo, J.G.A. Factors influencing the use of deep learning for plant disease recognition. *Biosyst. Eng.* **2018**, *172*, 84–91. <https://doi.org/10.1016/j.biosystemseng.2018.05.013>
8. Brahimi, M.; Arsenovic, M.; Laraba, S.; Sladojevic, S.; Boukhalfa, K.; Moussaoui, A. Deep learning for plant diseases: Detection and saliency map visualisation. In *Human and Machine Learning*; Human–Computer Interaction Series; Springer: Cham, Switzerland, 2018; pp. 93–117. https://doi.org/10.1007/978-3-319-90403-0_6
9. Singh, D.; Jain, N.; Jain, P.; Kayal, P.; Kumawat, S.; Batra, N. PlantDoc: A dataset for visual plant disease detection. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, Hyderabad, India, 5–7 January 2020; pp. 249–253. <https://doi.org/10.1145/3371158.3371196>
10. Toda, Y.; Okura, F. How convolutional neural networks diagnose plant disease. *Plant Phenomics* **2019**, *2019*, 9237136. <https://doi.org/10.34133/2019/9237136>
11. Wei, T.; Chen, Z.; Huang, Z.; Yu, X. Benchmarking in-the-wild multimodal disease recognition and a versatile baseline. In *Proceedings of the 32nd ACM International Conference on Multimedia*, Melbourne, Australia, 28 October–1 November 2024; pp. 1593–1601. <https://doi.org/10.1145/3664647.3680599>
12. Kolesnikov, N.S. Class-weighted CORAL and adversarial domain adaptation for cross-domain plant disease recognition: A comparative study. *Comput. Electron. Agric.* **2026**, submitted.
13. Thapa, R.; Zhang, K.; Snively, N.; Belongie, S.; Khan, A. The Plant Pathology Challenge 2020 data set to classify foliar disease of apples. *Appl. Plant Sci.* **2020**, *8*, e11390. <https://doi.org/10.1002/aps3.11390>

14. Wilson, G.; Cook, D.J. A survey of unsupervised deep domain adaptation. *ACM Trans. Intell. Syst. Technol.* **2020**, *11*, 1–46. <https://doi.org/10.1145/3400066>
15. Sun, B.; Saenko, K. Deep CORAL: Correlation alignment for deep domain adaptation. In *Computer Vision – ECCV 2016 Workshops*; Lecture Notes in Computer Science, Vol. 9915; Springer: Cham, Switzerland, 2016; pp. 443–450. https://doi.org/10.1007/978-3-319-49409-8_35
16. Long, M.; Cao, Y.; Cao, Z.; Wang, J.; Jordan, M.I. Transferable representation learning with deep adaptation networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2019**, *41*, 3071–3085. <https://doi.org/10.1109/TPAMI.2018.2868685>
17. Ganin, Y.; Ustinova, E.; Ajakan, H.; Germain, P.; Larochelle, H.; Laviolette, F.; Marchand, M.; Lempit-sky, V. Domain-adversarial training of neural networks. *J. Mach. Learn. Res.* **2016**, *17* (59), 1–35. <http://jmlr.org/papers/v17/15-239.html>
18. Tzeng, E.; Hoffman, J.; Saenko, K.; Darrell, T. Adversarial discriminative domain adaptation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 21–26 July 2017; pp. 2962–2971. <https://doi.org/10.1109/CVPR.2017.316>
19. Long, M.; Cao, Z.; Wang, J.; Jordan, M.I. Conditional adversarial domain adaptation. In *Advances in Neural Information Processing Systems 31*; Curran Associates: Red Hook, NY, USA, 2018. <https://proceedings.neurips.cc/paper/2018/hash/ab88b15733f543179858600245108dd8-Abstract.html>
20. Jin, Y.; Wang, X.; Long, M.; Wang, J. Minimum class confusion for versatile domain adaptation. In *Computer Vision – ECCV 2020*; Lecture Notes in Computer Science, Vol. 12366; Springer: Cham, Switzerland, 2020; pp. 464–480. https://doi.org/10.1007/978-3-030-58589-1_28
21. Li, Y.; Wang, N.; Shi, J.; Hou, X.; Liu, J. Adaptive batch normalization for practical domain adaptation. *Pattern Recognit.* **2018**, *80*, 109–117. <https://doi.org/10.1016/j.patcog.2018.03.005>
22. Wu, X.; Fan, X.; Luo, P.; Choudhury, S.D.; Tjahjadi, T.; Hu, C. From laboratory to field: Unsuper-vised domain adaptation for plant disease recognition in the wild. *Plant Phenomics* **2023**, *5*, 0038. <https://doi.org/10.34133/plantphenomics.0038>
23. Jeon, W.; Kim, T.; Choi, S.; Yang, K.; Kim, S.Y.; Song, M. Bridging the lab-to-field gap in plant disease diagnosis through unsupervised domain adaptation enhanced by background recombination. *Ecol. Inform.* **2026**, *93*, 103579. <https://doi.org/10.1016/j.ecoinf.2025.103579>
24. Tunio, M.H.; Li, J.; Zeng, X.; Ahmed, A.; Shah, S.A.; Shaikh, F.A.; Mallah, G.A.; Yahya, A. Advan-cing plant disease classification: A robust and generalized approach with transformer-fused con-volution and Wasserstein domain adaptation. *Comput. Electron. Agric.* **2024**, *227*, 109574. <https://doi.org/10.1016/j.compag.2024.109574>
25. Salman, Z.; Muhammad, A.; Han, D. Plant disease classification in the wild using vision transformers and mixture of experts. *Front. Plant Sci.* **2025**, *16*, 1522985. <https://doi.org/10.3389/fpls.2025.1522985>
26. Huan, Y.; Chen, Y.; Zhou, X. A unified self-supervised framework for plant disease detection on labor-atory and in-field images. *Electronics* **2025**, *14*, 3410. <https://doi.org/10.3390/electronics14173410>
27. Navarro, G. A guided tour to approximate string matching. *ACM Comput. Surv.* **2001**, *33*, 31–88. <https://doi.org/10.1145/375360.375365>
28. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 3–7 November 2019; pp. 3980–3990. <https://doi.org/10.18653/v1/D19-1410>
29. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; Kalenichenko, D. Quantiz-ation and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 18–23 June 2018; pp. 2704–2713. <https://doi.org/10.1109/CVPR.2018.00286>
30. Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv* **2018**, arXiv:1806.08342. <https://doi.org/10.48550/arXiv.1806.08342>
31. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv* **2017**,

arXiv:1704.04861. <https://doi.org/10.48550/arXiv.1704.04861>

32. Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.-C. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 18–23 June 2018; pp. 4510–4520. <https://doi.org/10.1109/CVPR.2018.00474>
33. Tan, M.; Le, Q.V. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 9–15 June 2019; PMLR 97, pp. 6105–6114. <https://proceedings.mlr.press/v97/tan19a.html>
34. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
35. Efron, B.; Tibshirani, R.J. *An Introduction to the Bootstrap*; Chapman & Hall/CRC: Boca Raton, FL, USA, 1994. <https://doi.org/10.1201/9780429246593>